

Step 1 — Writer Drafting Chain — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

`agents/writer_chain.py`

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

def build_writer_chain():
    prompt = ChatPromptTemplate.from_template(
        "Write a short draft based on these research notes:\n{notes}"
    )
    model = ChatOpenAI(model="gpt-4o-mini")
    parser = StrOutputParser()
    return prompt | model | parser
```

`main.py`

```
from agents.writer_chain import build_writer_chain

fake_notes = [
    "Electric bikes cost $800-3000. Battery range 20-60 miles. Growing 15%/year."
    "Topic: remote work. Notes: productivity mixed, saves commute time, isolation"
    "Topic: coffee. Notes: arabica vs robusta, cold brew trend, price volatility."
]

chain = build_writer_chain()
for notes in fake_notes:
    draft = chain.invoke({"notes": notes})
    print("---")
    print(draft)
```

This works and is a reasonable first version. It's missing type hints, there's no `if __name__ == "__main__":` guard, and nothing stops it from being called with empty notes — all fine to skip on a genuinely first pass.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with a real test endpoint

agents/writer_chain.py

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable

WRITER_PROMPT = (
    "You are a content writer. Write a short, clear draft (3-5 paragraphs) "
    "based only on these research notes. Do not invent facts that aren't "
    "in the notes.\n\nResearch notes:\n{notes}"
)

def build_writer_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_template(WRITER_PROMPT)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
    parser = StrOutputParser()
    return prompt | model | parser
```

main.py

```
from agents.writer_chain import build_writer_chain

FAKE_NOTES: list[str] = [
    "Electric bikes cost $800-3000. Battery range 20-60 miles. Growing 15%/year."
    "Topic: remote work. Notes: productivity mixed, saves commute time, isolation"
    "Topic: coffee. Notes: arabica vs robusta, cold brew trend, price volatility."
]

def main() -> None:
    chain = build_writer_chain()
    for notes in FAKE_NOTES:
        draft = chain.invoke({"notes": notes})
        print("---")
```

```
print(draft)

if __name__ == "__main__":
    main()
```

Difference from Basic: the prompt is pulled out into its own constant (`WRITER_PROMPT`) instead of buried inline — makes it easy to find and tune later, which matters once Step 2 needs a second, similar prompt for redrafting with feedback. Full type hints throughout. The prompt itself is more specific — "don't invent facts not in the notes" matters a lot once Research (Step 3) starts feeding this chain real, limited information; a vague prompt tends to quietly hallucinate details to fill gaps. The `if __name__ == "__main__":` guard means `main.py` can later be imported without immediately running its test calls. Nothing here yet handles empty input or strengthens the "don't invent facts" instruction beyond one sentence in a template string — that's what Advanced adds.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — guard clause against empty input

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable

WRITER_PROMPT = (
    "You are a content writer. Write a short, clear draft (3-5 paragraphs) "
    "based only on these research notes. Do not invent facts that aren't "
    "in the notes.\n\nResearch notes:\n{notes}"
)

def build_writer_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_template(WRITER_PROMPT)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
    return prompt | model | StrOutputParser()

def run_writer_chain(chain: Runnable, notes: str) -> str:
    if not notes or not notes.strip():
        raise ValueError("notes cannot be empty – nothing to draft from")
    return chain.invoke({"notes": notes})
```

This catches the case Basic and Intermediate both miss: calling the chain with `notes=""` no longer burns an API call to produce a fluent, content-free draft — it fails immediately, with a message that says exactly what's wrong, right where the bad input entered the system.

Approach 2 — system/human split for a stickier instruction

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable

WRITER_SYSTEM_PROMPT = (
```

```

    "You are a content writer. Write a short, clear draft (3-5 paragraphs) "
    "based only on the research notes you are given. Do not invent facts "
    "that aren't in the notes."
)
WRITER_TEMPERATURE = 0.7

def build_writer_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_messages([
        ("system", WRITER_SYSTEM_PROMPT),
        ("human", "Research notes:\n{notes}"),
    ])
    model = ChatOpenAI(model="gpt-4o-mini", temperature=WRITER_TEMPERATURE)
    return prompt | model | StrOutputParser()

def run_writer_chain(chain: Runnable, notes: str) -> str:
    if not notes or not notes.strip():
        raise ValueError("notes cannot be empty – nothing to draft from")
    return chain.invoke({"notes": notes})

if __name__ == "__main__":
    test_chain = build_writer_chain()
    for test_notes in [
        "Electric bikes cost $800-3000. Battery range 20-60 miles. Growing 15%/ye",
        "Topic: remote work. Notes: productivity mixed, saves commute time, isola",
        "Topic: coffee. Notes: arabica vs robusta, cold brew trend, price volatil",
    ]:
        print("---")
        print(run_writer_chain(test_chain, test_notes))

    try:
        run_writer_chain(test_chain, " ")
    except ValueError as e:
        print("Correctly rejected empty notes:", e)

```

Expected output: three drafts, then `Correctly rejected empty notes: notes cannot be empty – nothing to draft from`.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's chain works correctly as long as every caller happens to pass real notes. Approach 1 adds a safety net for when that assumption breaks — it doesn't change the prompt at all, just wraps the chain with an input check. Approach 2 is a separate improvement layered on top: moving the instruction into a dedicated `system` message, which models tend to follow more reliably than one instruction sentence mixed into a long user-role template — worth doing specifically because "don't invent

facts" is a correctness requirement here, not a style preference. Both are small, additive changes; a real version would keep both together, which is what the combined code above does.

Which one should you actually write? Keep both. Approach 1's guard clause costs three lines and catches a real bug class (silent, confusing failures three agents deep in Step 5) before it can happen. Approach 2's system/human split costs nothing extra at runtime and makes the "no invented facts" rule more likely to actually hold once Step 3's Research agent starts feeding this chain real, limited information instead of your generous made-up test notes. Neither is over-engineering for a project whose entire premise is "prove this piece is trustworthy before a team depends on it."